

```
print r.lrange("list1",0,-1)
print "pop from right ", r.rpop('list1')
print "pop from left ",r.lpop('list1')
print r.lrange("list1",0,-1)
```

The above program is very easy to understand. The results we have achieved in the above program are the same as what we achieved in *redis-cli* mode. See the output in Figure 6.

While there are a lot of advantages of using Redis, there are some disadvantages too. You should have more memory than your data requires, because Redis forks on each snapshot dump, which can consume extra memory. Also, you can't roll back a transaction.

In programming, you must always take care of the number of connections. Sometimes people are lazy while programming, leading to the 'max client reached problem'.

Alternatives to Redis

Well, this totally depends upon the situation. If you just need fast retrieval, you can use Memcache, but this does not store data on the disk. NoSQL databases like MongoDB and Couchbase might be alternatives for some use cases.

You can compare the Redis alternatives at:

- <https://www.g2crowd.com/products/redis/competitors/alternatives>

Continued Page from 36...

To delete files using PHP-MongoDB-GridFS, use the script given below:

```
<?php
$BigDataConnection = new Mongo("127.0.0.1:27017");
$mydb = $BigDataConnection->BigDatabase;
$mydb->authenticate("database-username","database-
password");
$biggrid = $db->getGridFS();
$myfilename = $_REQUEST["myfile"];
$myfile = $biggrid->findOne($myfilename);
$myid = $file['_id'];
$biggrid->delete($myid);
$BigDataConnection->close();
exit(0);
?>
```

Interfacing MongoDB-GridFS with Python

To integrate Python with MongoDB-GridFS, attach the *pymongo* library with the existing Python set-up. After this step, GridFS can be used.

On the Python prompt, the following instructions can be executed for Python-MongoDB-GridFS interfacing:

```
>>> from pymongo import MongoClient
>>> import gridfs
```

```
File Edit View Search Terminal Help
[root@example ~]# python redis1.py
1
2
3
[root@example ~]# █
```

Figure 5: Output of *redis1.py*

```
[root@mohit ~]# python redis3.py
['one', 'two', 'three', 'four']
pop from right four
pop from left one
['two', 'three']
[root@mohit ~]# vim redis3.py
[root@mohit ~]# █
```

Figure 6: Output of *redis3.py*

- <https://hazelcast.com/use-cases/nosql/redis-replacement/>
It is recommended that you also read up on Redis commands and Redis Sentinel connections. 

By: Mohit

The author is a certified ethical hacker and EC Council certified security analyst. He has a master's in computer science from Thapar University, and is the author of 'Python Penetration Testing Essentials'. You can contact him at mohitraj.cs@gmail.com and <https://in.linkedin.com/in/mohit-raj-990a852a>.

For selecting database and file system, execute the following instructions:

```
>>> mydb = MongoClient().MyGridFS
>>> filesystem = gridfs.GridFS(mydb)
```

To insert a new file, *put()* is used:

```
>>> myfile = filesystem.put("My New File")
```

To read the file contents, *get()* is used:

```
>>> filesystem.get(myfile).read()
```

These executions can be used for assorted applications involving Big Data processing. MongoDB-GridFS interfacing can also be done with other programming languages with higher levels of integrity and performance, including Java, C++, JavaScript, Ruby, Scala, Haskell and many others. 

By: Dr Gaurav Kumar

The author is the MD of Magma Research and Consultancy Pvt Ltd, Ambala City. He is associated with various academic and research institutes, where he delivers lectures and conducts technical workshops. He can be contacted at kumargaurav.in@gmail.com.